

Modelling Shared-Space Coordination in mCRL2: a Bach-to-mCRL2 Translation Framework

Corentin Reuther Jean-Marie Jacquet

Nadi Research Institute
Faculty of Computer Science, University of Namur
Belgium

{corentin.reuther,jean-marie.jacquet}@unamur.be

Although significant research has focused on the theory and implementation of data-based coordination languages, the critical aspect of their automated verification using model-checking techniques remains underexplored, which is essential for ensuring reliability and correctness in distributed systems. Existing tools, such as Anemone, provide a solid foundation for reachability-based verification of Bach programs. While they effectively analyze properties expressed in terms of state attainability, extending support to more expressive temporal specifications—such as liveness properties or invariants over shared space contents—remains an open opportunity. Such extensions are important to capture comprehensive system behaviors, for instance, ensuring that “a request is always matched by a response” or that “no message is silently lost”. Addressing this limitation, we propose an automated translation from Bach to mCRL2, which explicitly represents the shared space, thereby enabling the use of mCRL2’s μ -calculus model checker to verify complex properties beyond simple reachability. Complementing this translation, we introduce a systematic method to analyze the shared space in mCRL2 concerning data reachability, by directly expressing properties over the shared space’s contents in the μ -calculus, thus providing a clearer framework for verification. This approach enables a novel verification process that combines action-based properties with state-based properties over the shared space contents, a largely unexplored area in current coordination-language verification approaches, offering a new dimension of analysis.

1 Introduction and Related Work

In the era of distributed computing, systems confront unprecedented challenges in managing interactions between autonomous components. These systems, where human and technical elements coexist, often exhibit emergent behaviors that can lead to failures if not properly coordinated. For instance, in a microservices setup, a single faulty interaction can cascade into system-wide service disruptions, as illustrated by a major Amazon Web Services outage in October 2025 [17]. In light of this complexity, it appears imperative to use robust coordination models that separate computation and interaction, in order to reason and verify the behavior of systems in a formal way.

The concept of coordination languages originated with Gelernter and Carriero, who introduced Linda in 1985 [9] as a generative communication model based on a shared tuple space — a logical blackboard where processes deposit and retrieve data without direct coupling [4, 8].

Bach [1, 5, 12, 13], developed at the University of Namur, is directly inspired by Linda and the algebra of communicating processes (ACP) [3] to propose a modern coordination language. It combines ACP operators (sequential $;$, parallel $||$, non-deterministic choice $+$ operators) with Linda-like primitives (*tell*, *get*, *ask*, *nask*) operating on a shared space.

While Bach excels at modeling such interactions, verifying its behavioral properties, such as fairness or absence of losses, has proved challenging with the built-in model checker of Anemone [12], a veri-

fication platform developed for Bach. Writing complex temporal properties in this environment turned out to be cumbersome, as the tool primarily focuses on state reachability within the shared space.

This limitation prompted the exploration of mCRL2 [10], a process-algebra-based tool that employs the μ -calculus for rigorous verification and is equipped with state-of-the-art model checking techniques. While Groote and Mousavi [10] provide a comprehensive framework for modelling and verifying channel-based communicating systems, Bach processes communicate through a shared space whose contents must themselves be subject to verification — an aspect not addressed by standard mCRL2 methodology.

The question of formally verifying coordination languages based on shared spaces is not new, but remains largely open in its most expressive forms. De Nicola et al. [7] tackle a related challenge with STOcKLAIM, a stochastic extension of KLAIM [6]: they model action durations as exponential distributions, translate systems into Continuous Time Markov Chains, and verify quantitative properties such as the probability of reaching a given state within a time bound. This is a genuinely different angle — their concern is *how fast* things happen, while ours is *whether* they happen at all. More fundamentally, their atomic propositions track process locations, not data presence in a shared space. To our best knowledge, directly reasoning over the *contents* of a shared space as part of a temporal verification framework remains unexplored — and this is precisely the gap we begin to explore.

This paper develops an automated tool for this translation, available as an artefact in [18]. The tool makes the shared space explicit as a parallel process in mCRL2 and encodes Bach primitives as actions, enabling mCRL2 to effectively model Linda-like coordination structures. A key contribution of this paper is extending mCRL2’s μ -calculus proofs to incorporate shared content alongside actions, enabling content-aware verification (e.g., querying the presence of specific elements or data values in the shared space) beyond standard action-based checks. More precisely, the objectives of our work are twofold: on the one hand, to describe the Bach-to-mCRL2 translation, highlighting how observable behavior is preserved and the shared space is explicitly modelled; and, on the other hand, to lay the groundwork for verification of temporal properties, combining action-based and state-based reasoning, preparing for future μ -calculus proofs over shared space contents.

The remainder of this paper is organized in five sections. Section 2 introduces the Bach coordination language, its primitives, composition operators, and trace semantics. Section 3 presents the Bach-to-mCRL2 translation, including the explicit encoding of the shared space, the mapping of primitives to actions, and the trace correspondence theorem. Section 4 introduces the verification framework, recalling the temporal operators of Anemone and the μ -calculus fragment of mCRL2, and presenting the systematic translation of Bach temporal properties into μ -calculus formulas, including the content-aware approach based on Blackboard instrumentation. Section 5 illustrates the approach with a synchronous Load Balancer case study, exploring how the translation enables the verification of coordination properties combining action-based and content-aware reasoning. Finally, Section 6 concludes with a discussion of the results and perspectives for future work.

2 Bach Coordination Language: Syntax and Semantics

In order to make the paper as self-contained as possible, taking inspiration from [1, 2, 5, 12, 13], this section presents the key features of the Bach coordination language, providing the necessary foundation for its translation to mCRL2 and formal verification. Our goal is to define its syntax, operational semantics, and the observable behavior of agents through traces, distinguishing clearly between observable actions (Definition 1) and structural constructs (Definition 2), which are formally defined later in this section.

Combining Linda's generative communication model with the Algebra of Communicating Processes (ACP), Bach enables agents to interact exclusively through a *shared space*. Agents use primitives for insertion, removal, and observation, allowing a decoupled approach that is ideal for modeling asynchronous interactions in distributed systems.

Scope: This work addresses the core Bach coordination language without extensions such as active data (Multi-Bach), AnimBach, or guarded lists, which address different concerns. The core primitives (`tell`, `ask`, `nask`, `get`) combined with ACP operators form a sufficient base for distributed coordination verification in mCRL2. The syntax and semantics presented afterwards are adapted from the formal definition in [2] for the scope and notational conventions of this paper; the underlying language and its semantics are unchanged.

2.1 Syntax

The syntax of Bach is structured in five conceptual layers, ranging from data representation to process composition. While these layers provide a syntactic view of the language, our presentation is intentionally oriented towards its operational interpretation, as developed in Section 2.2. In particular, rather than focusing solely on grammatical well-formedness, we emphasize the constructs that have a direct impact on execution and observable behavior, which will be essential for the definition of trace semantics and the correspondence theorem (Theorem 1).

This layered presentation is therefore not purely syntactic: it reflects the structure of the operational semantics by separating data, communication primitives, control structures, and agent-level composition constructs. This separation will play a central role in the definition of traces and in the translation into mCRL2, where only primitive communication actions are retained as observable behavior.

Data layer. The data layer of Bach distinguishes two kinds of identifiers and a notion of structured information term:

$$\begin{aligned} lid &::= \text{string of letters and digits starting with a lowercase letter} \\ uid &::= \text{string of letters and digits starting with an uppercase letter} \\ si &::= lid \mid lid(si_1, \dots, si_n) \end{aligned} \tag{1}$$

A *lid* (lowercase identifier) used on its own is called a **token** in the literature and represents atomic data (e.g., `request`, `order`). When a *lid* occurs as the head of a structured term $lid(si_1, \dots, si_n)$, we call it a **functor**. A *uid* (uppercase identifier) denotes an identifier used as a procedure name (e.g., `ProducerAgent`, `LoadBalancer`, see Section 2.1). A *si* (*si-term*, short for *structured information term*) is therefore either a token, or a structured term obtained by applying a functor to a sequence of si-terms (e.g., `request(producer, consumer, id, content)`).

Remark 1. In the remainder of this paper, we frequently refer to the notion of a *shared space*, which is modelled as a multiset of structured information terms (si-terms). In the coordination literature, this abstraction is often referred to as a *tuple space*, and si-terms are commonly called *tuples*. These notions are considered equivalent in this work.

Communication primitives. Bach provides four basic communication primitives (`cprim`) operating on si-terms:

$$cprim ::= \text{tell}(si) \mid \text{ask}(si) \mid \text{nask}(si) \mid \text{get}(si) \tag{2}$$

Their meanings: `tell(si)` inserts an occurrence of *si* into the shared space (always succeeds); `ask(si)` checks for the presence of a matching *si* (blocks if absent); `nask(si)` checks for the absence (blocks if present); and `get(si)` removes a matching *si* (blocks if absent).

Control structures. Conditions are built from comparisons over *si*-terms and combined with Boolean operators:

$$\begin{aligned} compop &::= < \mid \leq \mid > \mid \geq \mid = \mid \neq \\ boolop &::= \& \mid \mid \mid ! \\ c &::= si_1 compop si_2 \mid c_1 boolop c_2 \end{aligned} \quad (3)$$

Agents and Procedures. Agents are the active entities of Bach. They are built from communication primitives and structural constructs (Definition 2). Communication primitives correspond to observable actions (Definition 1) on the shared space, while structural constructs define how agents are composed and controlled. The grammar of agents is:

$$\begin{aligned} ag &::= cprim \mid ag_1; ag_2 \mid ag_1 \parallel ag_2 \mid ag_1 + ag_2 \mid \\ &\quad c \rightarrow ag_1 \diamond ag_2 \mid \text{sum } lid \text{ in } D : ag \mid pn(si_1, \dots, si_n) \end{aligned} \quad (4)$$

The composition operators have their usual meaning. Sequential composition $ag_1; ag_2$ runs the first agent and then the second; parallel composition $\parallel$ allows interleaved execution; non-deterministic choice $+$ selects one of the two branches.

In the guarded choice $c \rightarrow ag_1 \diamond ag_2$, the condition c may refer to variables already in scope; if it evaluates to true, ag_1 executes, otherwise ag_2 . The quantification $\text{sum } lid \text{ in } D : ag$ ranges over all values of the finite domain D (Section 2.1) assigned to lid , and behaves as a non-deterministic choice between the corresponding instantiations of ag . Lastly, the grammar includes procedure calls $pn(si_1, \dots, si_n)$, which invoke the procedure named pn by substituting the *si*-term arguments $si_1, \dots, si_n$ for its parameters and executing its body.

A procedure is introduced by a *procedure equation* of the form:

$$pn(si_1, \dots, si_n) = ag \quad (5)$$

where pn is an uppercase identifier (a *uid*) denoting the procedure name, and the si_i are parameters in scope inside the body ag . The keyword `proc` is used at the top level of a Bach specification to introduce one or more such procedure equations.

Procedures are not agents themselves, but definitions that can be invoked by agents through procedure calls. As shown in the grammar of agents (Equation 4), procedure calls may appear inside agents and support recursive definitions.

Domains. In this paper, domains are treated as finite sets of constants. For instance, in the case study (Section 5), we define `MyInteger = {zero, one, two}`. In the full algebraic specification of Bach [2], such a domain is declared using a `sort` declaration (e.g., `sort MyInteger = struct zero | one | two`), together with optional `map` and `eqn` declarations to define functions over these data types. More generally, `sort` introduces new data types, `map` defines function symbols over these types, and `eqn` provides their defining equations.

In this work, we abstract from this algebraic specification and reason directly over finite sets of values. This abstraction is sufficient for our results, since Theorem 1 (Trace Correspondence) depends

only on observable actions and process composition, and not on the internal algebraic structure of data types.

2.2 Operational Semantics

In order to present Bach's operational semantics we use labelled transition systems like other papers in the field [2]. We use configurations of the form of $\langle A | \sigma \rangle$, where A is an agent (i.e., an instance of ag as defined in Equation 4) and σ is the shared space, modelled as a multiset of si-terms. Transitions represent computation steps, with rules for primitives and operators derived from ACP [3] and Linda-like semantics [4, 9].

The operational semantics of Bach distinguishes two complementary categories of language constructs: those that produce *observable actions* on the shared space, and those that govern the *structural composition* of agents. We make this distinction formal below, together with the notion of a *terminated agent* representing the result of a successful execution.

Definition 1 (Observable actions). *The observable actions of Bach are the labels produced by the execution of communication primitives. They are denoted by the label α ranging over the four possible labels: t^+ for $tell(t)$, t^- for $get(t)$, $t^?$ for $ask(t)$, and $t^\sim$ for $nask(t)$, where t is a si-term.*

Definition 2 (Structural constructs). *The structural constructs of Bach are the operators that compose agents without producing observable actions on their own: sequential composition ($;$), parallel composition ($\parallel$), non-deterministic choice ($+$), guarded choice ($\rightarrow \diamond$), quantification (sum), and procedure calls. Their behavior is defined inductively from the observable actions of their sub-agents.*

Definition 3 (Terminated agent). *We extend the syntax of agents (Equation 4) with a distinguished constant E , called the terminated agent, representing successful completion. By convention, E admits no outgoing transition: for any shared space σ and any label α , $\langle E | \sigma \rangle \nrightarrow$.*

The transition rules of Bach are presented in two parts: rules governing the communication primitives (Section 2.2.1), which produce observable actions, and rules governing the structural constructs (Section 2.2.2), which combine sub-agents inductively. This separation is central to the trace-based translation into mCRL2 developed in Section 3, where only observable actions are retained in the observable behavior of an agent.

2.2.1 Transitions rules for Primitives

Figure 1 specifies the four transition rules governing how Bach primitives interact with the shared space. The rules naturally split into two pairs: the first pair modifies the shared space, while the second pair only observes it.

The first pair captures the primitives that *modify* the shared space. Rule (T) states that $tell(t)$ always succeeds and adds an occurrence of t to the shared space, producing the labelled transition t^+ . Conversely, rule (G) describes $get(t)$, which removes one occurrence of t and produces the label t^- . This rule applies only when t is already present in the shared space; otherwise, the agent blocks until another agent inserts it.

The second pair captures the primitives that *observe* the shared space without modifying it. Rule (A) describes $ask(t)$: it requires t to be present, observes it without consuming it, and produces the label $t^?$. Its counterpart, rule (N), describes $nask(t)$: it has the explicit premise $t \notin \sigma$ — that is, it requires the *absence* of t — and produces the label $t^\sim$.

$$\begin{array}{ll}
(T) \quad \langle \text{tell}(t) \mid \sigma \rangle \xrightarrow{t^+} \langle E \mid \sigma \cup \{t\} \rangle & (A) \quad \langle \text{ask}(t) \mid \sigma \cup \{t\} \rangle \xrightarrow{t^?} \langle E \mid \sigma \cup \{t\} \rangle \\
(G) \quad \langle \text{get}(t) \mid \sigma \cup \{t\} \rangle \xrightarrow{t^-} \langle E \mid \sigma \rangle & (N) \quad \frac{t \notin \sigma}{\langle \text{nask}(t) \mid \sigma \rangle \xrightarrow{t^\sim} \langle E \mid \sigma \rangle}
\end{array}$$

Figure 1: Transition rules for the primitives in Bach [2].

$$\begin{array}{ll}
(S) \quad \frac{\langle A \mid \sigma \rangle \xrightarrow{\alpha} \langle A' \mid \sigma' \rangle}{\langle A; B \mid \sigma \rangle \xrightarrow{\alpha} \langle A'; B \mid \sigma' \rangle} & (P) \quad \frac{\langle A \mid \sigma \rangle \xrightarrow{\alpha} \langle A' \mid \sigma' \rangle}{\langle A \parallel B \mid \sigma \rangle \xrightarrow{\alpha} \langle A' \parallel B \mid \sigma' \rangle} \\
& \frac{\langle B \mid \sigma \rangle \xrightarrow{\alpha} \langle B' \mid \sigma' \rangle}{\langle B \parallel A \mid \sigma \rangle \xrightarrow{\alpha} \langle B' \parallel A \mid \sigma' \rangle} \\
(C) \quad \frac{\langle A \mid \sigma \rangle \xrightarrow{\alpha} \langle A' \mid \sigma' \rangle}{\langle A + B \mid \sigma \rangle \xrightarrow{\alpha} \langle A' \mid \sigma' \rangle} & (Co) \quad \frac{\models C, \langle A \mid \sigma \rangle \xrightarrow{\alpha} \langle A' \mid \sigma' \rangle}{\langle C \rightarrow A \diamond B \mid \sigma \rangle \xrightarrow{\alpha} \langle A' \mid \sigma' \rangle} \\
& \frac{\langle B \mid \sigma \rangle \xrightarrow{\alpha} \langle B' \mid \sigma' \rangle}{\langle B + A \mid \sigma \rangle \xrightarrow{\alpha} \langle B' \mid \sigma' \rangle} & \frac{\langle !C \rightarrow B \diamond A \mid \sigma \rangle \xrightarrow{\alpha} \langle A' \mid \sigma' \rangle}{} \\
(Pc) \quad \frac{P(\bar{x})=A, \langle A[\bar{x}/\bar{u}] \mid \sigma \rangle \xrightarrow{\alpha} \langle A' \mid \sigma' \rangle}{\langle P(\bar{u}) \mid \sigma \rangle \xrightarrow{\alpha} \langle A' \mid \sigma' \rangle} & (Sum) \quad \frac{d \in D, \langle A[x/d] \mid \sigma \rangle \xrightarrow{\alpha} \langle A' \mid \sigma' \rangle}{\langle \text{sum } x \text{ in } D : A \mid \sigma \rangle \xrightarrow{\alpha} \langle A' \mid \sigma' \rangle}
\end{array}$$

Figure 2: Operational semantics rules for the structural constructs of Bach. All rules except (Sum) follow [2]; (Sum) is introduced in this work.

2.2.2 Transition Rules for Structural Constructs

The structural constructs of Bach define how agents are composed and how control flows during execution. Their operational semantics follow an interleaving model inspired by ACP and is given in Figure 2. Rule (S) handles sequential composition $A; B$: the first agent A executes, and the continuation B remains pending. Rule (P) handles parallel composition $A \parallel B$: either side may make a transition, yielding interleaved executions. Rule (C) handles non-deterministic choice $A + B$: either branch may evolve, with the unselected branch discarded. Rule (Co) handles guarded choice $C \rightarrow A \diamond B$: when the condition C holds, A is executed; when it does not, B is. Rule (Pc) handles procedure calls $P(\bar{u})$: the call is unfolded by substituting the arguments into the procedure body A . Rule (Sum) handles the quantification operator $\text{sum } x \text{ in } D : A$: any value $d \in D$ may be selected, instantiating A accordingly.

Remark 2 (On the multi-conclusion formulation). The rules (P), (C), and (Co) are presented with two symmetric conclusions to capture the commutativity of $\parallel$, $+$, and $\rightarrow \diamond$ respectively. An alternative formulation would introduce a structural congruence relation $\equiv$ on agents (e.g., $A \parallel B \equiv B \parallel A$) together with single-conclusion rules.

2.3 Trace Semantics of Bach

Definition 4 (Trace semantics of Bach). *We define the observational semantics of a Bach agent as the function $\mathcal{O}_B : \text{Agent} \rightarrow \mathcal{P}(\text{Trace})$, where a trace is a finite sequence of observable actions. We denote by $\mathcal{P}(\text{Trace})$ the power set of all such traces, i.e., the set of all possible (non-deterministic) execution*

behaviors of a given agent. For a Bach agent A , we define $\mathcal{O}_B(A)$ as follows:

$$\mathcal{O}_B(A) = \left\{ \alpha_1 \cdots \alpha_n \mid \langle A, \emptyset \rangle \xrightarrow{\alpha_1} \langle A_1, \sigma_1 \rangle \xrightarrow{\alpha_2} \cdots \xrightarrow{\alpha_n} \langle A_n, \sigma_n \rangle \rightarrow \right\}.$$

3 Bach-to-mCRL2 Translation and Associated Tools

This section describes the mapping from Bach to mCRL2 and introduces an automated translation tool. The resulting encoding provides a basis for subsequent analysis in mCRL2, including the verification of temporal properties using the μ -calculus, which is developed later in this paper.

3.1 Overview of mCRL2

mCRL2 [10] is a process algebra-based formalism and toolset designed for modeling and verifying concurrent systems. Sharing its roots in the Algebra of Communicating Processes (ACP) with Bach, mCRL2 is particularly well-suited for this translation. Its distinguishing feature is the integration of the modal μ -calculus, a highly expressive logic that enables automated verification of complex behavioral properties.

The mCRL2 toolset provides simulation (`simulator`), state space generation (`lps2lts`), and property verification (`pbes2bool`) using μ -calculus proofs. These features allow a more comprehensive analysis of Bach models, enabling developers to explore agent interactions and verify system properties beyond what is possible with the current Anemone tool.

3.2 Operational Semantics of the mCRL2 Fragment

To establish the correctness of our translation, we first introduce the operational semantics of the relevant fragment of mCRL2, adapted from [10]. Similar to Bach, we represent system behavior using labelled transition systems (LTS), where transitions are labelled by actions α , and E denotes the terminated agent (following the convention introduced in Definition 3). Figure 3 presents the rules for the fragment we use.

Rule (T_{mCRL2}) gives the basic action behaviour: an action a executes and reaches the terminated state E . Rules (S_{mCRL2}) , (C_{mCRL2}) , (Co_{mCRL2}) , (Pc_{mCRL2}) , and (Sum_{mCRL2}) handle respectively sequential composition, non-deterministic choice, guarded choice $C \rightarrow A \diamond B$, process calls, and quantification operator, with semantics analogous to their Bach counterparts. Two notable differences with respect to Bach are nonetheless worth pointing out, in addition to minor syntactic variations (sequential composition uses the dot operator $A.B$ instead of the semicolon, and quantification follows the syntax $sum\ x : D. A$ instead of $sum\ x\ in\ D : A$).

The first difference concerns parallel composition. In addition to independent interleavings — captured by $(P_{mCRL2}^{L/R})$ as in Bach — the rule (P_{mCRL2}^{sync}) allows two parallel components to synchronise their actions according to a communication function γ , which maps pairs of synchronisable actions to a combined action. The function γ is defined explicitly via the `comm` construct of mCRL2 (see Figure 4); when $\gamma(\alpha, \beta)$ is undefined, no synchronisation is possible and the rule does not apply. This synchronisation mechanism is specific to mCRL2 and is central to our translation, as we will use it to model the interaction between agents and the Blackboard (Section 3.4).

The second difference is the `allow` operator, governed by rule (Al_{mCRL2}) . It does not merely hide actions outside the set S from external observation: actions not in S are entirely *prevented from occurring*. In other words, `allow( $S, A$ )` behaves like A except that any transition labelled with an action $\alpha \notin S$ is

$$\begin{array}{ll}
(T_{mCRL2}) & a \xrightarrow{\alpha} E \\
(S_{mCRL2}) & \frac{A \xrightarrow{\alpha} A'}{A.B \xrightarrow{\alpha} A'.B} \\
(P_{mCRL2}^{sync}) & \frac{A \xrightarrow{\alpha} A' \quad B \xrightarrow{\beta} B'}{A \parallel B \xrightarrow{\gamma(\alpha, \beta)} A' \parallel B'} \\
(CO_{mCRL2}) & \frac{\models C, A \xrightarrow{\alpha} A' \quad C \rightarrow A \diamond B \xrightarrow{\alpha} A' \quad !C \rightarrow B \diamond A \xrightarrow{\alpha} A'}{C \rightarrow A \diamond B \xrightarrow{\alpha} A'} \\
(Sum_{mCRL2}) & \frac{d \in D \quad A[x/d] \xrightarrow{\alpha} A'}{\sum x:D. A \xrightarrow{\alpha} A'} \\
(Al_{mCRL2}) & \frac{A \xrightarrow{\alpha} A' \quad \alpha \in S}{allow(S, A) \xrightarrow{\alpha} allow(S, A')} \\
(C_{mCRL2}) & \frac{A \xrightarrow{\alpha} A'}{A+B \xrightarrow{\alpha} A' \quad B+A \xrightarrow{\alpha} A'} \\
(P_{mCRL2}^{L/R}) & \frac{A \xrightarrow{\alpha} A' \quad B \xrightarrow{\beta} B'}{A \parallel B \xrightarrow{\alpha} A' \parallel B \quad A \parallel B \xrightarrow{\beta} A \parallel B'} \\
(PC_{mCRL2}) & \frac{P(\bar{x})=A \quad A[\bar{x}/\bar{u}] \xrightarrow{\alpha} A'}{P(\bar{u}) \xrightarrow{\alpha} A'}
\end{array}$$

Figure 3: Operational semantics rules for the relevant fragment of mCRL2, adapted from [10].

blocked at the source. This operator plays a key role in defining the observable semantics of the translated system.

3.3 Trace Semantics of mCRL2

Definition 5 (Trace semantics of mCRL2). *We define the observational semantics of an mCRL2 agent as the function $\mathcal{O}_M : Agent \rightarrow \mathcal{P}(Trace)$, where a trace is a finite sequence of observable actions. For an mCRL2 agent A , we define $\mathcal{O}_M(A)$ as follows:*

$$\mathcal{O}_M(A) = \left\{ \alpha_1 \cdots \alpha_n \mid A \xrightarrow{\alpha_1} A_1 \xrightarrow{\alpha_2} \cdots \xrightarrow{\alpha_n} A_n \not\rightarrow \right\}.$$

3.4 Implementation of the Shared Space in mCRL2

In a Bach system, a set of agents $A_1, A_2, \dots, A_n$ execute in parallel and interact exclusively through the shared space. They can be summarized as a single composed agent $A = A_1 \parallel A_2 \parallel \cdots \parallel A_n$. To faithfully translate this into mCRL2, we implement the shared space as an explicit parallel process, called the Blackboard. This design, illustrated in Figure 4, separates agent behavior from state management, preserving the decoupled coordination semantics of Bach. The initial system is defined as the parallel composition of the Blackboard process and the translated agent A , which corresponds to the mCRL2 encoding of the composed Bach agent $A_1 \parallel \cdots \parallel A_n$. The Blackboard is initialized with an empty multiset, denoted $\{ : \}$, which in mCRL2 represents the empty shared space $\emptyset$.

Each primitive in Bach (`tell`, `get`, `ask`, `nask`) is translated into an action emitted by the agent process. These actions synchronize with corresponding internal Blackboard actions, producing observable events:

- `tell(t)`: synchronizes with Blackboard action `tb(t)` and performs observable action `Told(t)`
- `get(t)`: synchronizes with `gb(t)` and performs observable action `Got(t)`


```

proc Blackboard(myBB: BB) =
  sum x: Elem.
    tb(x) . Blackboard(myBB + {x:1})
  + sum x: Elem.
    (count(x, myBB) > 0) -> gb(x) . Blackboard(myBB - {x:1})
  + sum x: Elem.
    (count(x, myBB) > 0) -> ab(x) . Blackboard(myBB)
  + sum x: Elem.
    (count(x, myBB) == 0) -> nab(x) . Blackboard(myBB);

init allow({Told, Got, Asked, Nasked},
  comm({
    tb | tell -> Told, gb | get -> Got,
    ab | ask -> Asked, nab | nask -> Nasked
  },
  Blackboard({:}) || A
));

```

Figure 4: Explicit implementation of a shared space in mCRL2

- ask(t): synchronizes with ab(t) and performs observable action Asked(t)
- nask(t): synchronizes with nab(t) and performs observable action Nasked(t)

This interaction is modelled in mCRL2 through a synchronization mechanism between the agent and the Blackboard, followed by a restriction of the resulting action set to the intended observable behavior (via the *allow* structure). The Blackboard maintains the multiset of si-terms internally and updates it according to agent actions, while the synchronized actions serve as the observable interface for verification.

3.5 Trace Semantics Revisited

To support the inductive proof of the trace correspondence theorem, we introduce a generalized version of the Bach trace semantics that takes the initial shared space as an explicit parameter.

Definition 6 (Generalized Trace Semantics of Bach, $\mathcal{O}'$). *Let A be a Bach agent and σ a shared space. We define:*

$$\mathcal{O}'(A)(\sigma) = \left\{ \alpha_1 \cdots \alpha_n \mid \langle A \mid \sigma \rangle \xrightarrow{\alpha_1} \langle A_1 \mid \sigma_1 \rangle \xrightarrow{\alpha_2} \cdots \xrightarrow{\alpha_n} \langle A_n \mid \sigma_n \rangle \rightarrow \right\}$$

where the condition $\langle A_n \mid \sigma_n \rangle \rightarrow$ includes the case $n = 0$, i.e., when $\langle A \mid \sigma \rangle \rightarrow$ itself, yielding the empty trace ε .

Note that the original trace semantics $\mathcal{O}_B(A)$ defined in Definition 4 corresponds to the special case $\mathcal{O}'(A)(\emptyset)$.

Definition 7 (Recursive Trace Semantics of Bach, $\mathcal{O}''$). *Let A be a Bach agent and σ a shared space. We define $\mathcal{O}''(A)(\sigma)$ recursively as:*

$$\mathcal{O}''(A)(\sigma) = \{ \varepsilon \mid \langle A \mid \sigma \rangle \rightarrow \} \cup \left\{ \alpha_1 \cdot h \mid \langle A \mid \sigma \rangle \xrightarrow{\alpha_1} \langle A_1 \mid \sigma_1 \rangle \text{ and } h \in \mathcal{O}''(A_1)(\sigma_1) \right\}$$

where ε denotes the empty trace. The term $\{ \varepsilon \}$ serves as the base case of the recursion: it is trivially included, corresponding to the case $\langle A \mid \sigma \rangle \rightarrow$, just as in $\mathcal{O}'$ where the empty trace arises implicitly when $n = 0$.

Proposition 1 (Equivalence of Trace Semantics). *For any Bach agent A and shared space σ :*

$$\mathcal{O}'(A)(\sigma) = \mathcal{O}''(A)(\sigma)$$

Proof. Simple verification. □

3.6 Translation Functions Between Trace Semantics

To formally relate the traces of a Bach agent to those of its mCRL2 translation, we introduce two functions that map observable actions between the two formalisms.

Definition 8 (Bach-to-mCRL2 action translation, $b2m$). *The function $b2m$ maps Bach observable actions to mCRL2 observable actions:*

$$\begin{aligned} b2m(t^+) &= Told(t) & b2m(t^-) &= Got(t) \\ b2m(t^?) &= Asked(t) & b2m(t^\sim) &= Nasked(t) \end{aligned}$$

It extends to traces as follows: $b2m(\alpha_1 \cdots \alpha_n) = b2m(\alpha_1) \cdots b2m(\alpha_n)$.

Definition 9 (mCRL2-to-Bach action translation, $m2b$). *The function $m2b$ maps mCRL2 observable actions to Bach observable actions:*

$$\begin{aligned} m2b(Told(t)) &= t^+ & m2b(Got(t)) &= t^- \\ m2b(Asked(t)) &= t^? & m2b(Nasked(t)) &= t^\sim \end{aligned}$$

It extends to traces as follows: $m2b(\beta_1 \cdots \beta_n) = m2b(\beta_1) \cdots m2b(\beta_n)$.

For conciseness, we introduce the following notation for the translated system:

Definition 10 (Translated system, Act_{tr}). *Let A be a Bach agent. We denote by $Act_{tr}(A)$ the mCRL2 process:*

$$\begin{aligned} Act_{tr}(A) &= allow(\{Told, Got, Asked, Nasked\}, \\ &\quad comm(\{tb \mid tell \rightarrow Told, gb \mid get \rightarrow Got, ab \mid ask \rightarrow Asked, nab \mid nask \rightarrow Nasked\}, \\ &\quad BB(\cdot) \parallel tr(A))) \end{aligned}$$

where $tr(A)$ is the mCRL2 translation of the Bach agent A , and $BB(\{:\})$ is the Blackboard initialized with the empty shared space.

The function tr is defined by structural induction on the grammar of agents (Equation 4), mapping each Bach primitive to its corresponding mCRL2 action and each composition operator to its mCRL2 counterpart.

Theorem 1 (Trace Correspondence). *Let A be a Bach agent and σ a shared space. Then:*

1. *For any trace $\alpha_1 \cdots \alpha_n \in \mathcal{O}'(A)(\sigma)$, there exists a trace $\beta_1 \cdots \beta_n \in \mathcal{O}_M(Act_{tr}(A))$ such that:*

$$\alpha_1 \cdots \alpha_n = m2b(\beta_1 \cdots \beta_n)$$

2. *For any trace $\beta_1 \cdots \beta_n \in \mathcal{O}_M(Act_{tr}(A))$, there exists a trace $\alpha_1 \cdots \alpha_n \in \mathcal{O}'(A)(\sigma)$ such that:*

$$\beta_1 \cdots \beta_n = b2m(\alpha_1 \cdots \alpha_n)$$

Proof. Both directions are proved by induction on the length of the trace, using the recursive characterization $\mathcal{O}''$ established in Proposition 1, which allows us to decompose any trace into its first action and a continuation.

DIRECTION 1 ($\mathcal{O}' \rightarrow \mathcal{O}_M$). We prove that for any trace $\alpha_1 \cdots \alpha_n \in \mathcal{O}'(A)(\sigma)$, the trace $b2m(\alpha_1 \cdots \alpha_n)$ belongs to $\mathcal{O}_M(Act_{tr}(A))$.

Base case. If $\alpha_1 \cdots \alpha_n = \varepsilon$, then $\langle A \mid \sigma \rangle \nrightarrow$, and correspondingly $Act_{tr}(A)$ cannot perform any observable action, so $\varepsilon \in \mathcal{O}_M(Act_{tr}(A))$.

Inductive step. Assume the property holds for all traces of length n . Consider a trace $\alpha_1 \cdots \alpha_{n+1} \in \mathcal{O}''(A)(\sigma)$. By definition of $\mathcal{O}''$, there exists a transition:

$$\langle A \mid \sigma \rangle \xrightarrow{\alpha_1} \langle A_1 \mid \sigma_1 \rangle$$

with $\alpha_2 \cdots \alpha_{n+1} \in \mathcal{O}''(A_1)(\sigma_1)$. The translation of this primitive or structural step produces a corresponding transition in $Act_{tr}(A)$ with label $b2m(\alpha_1)$. By the induction hypothesis applied to A_1 , the suffix $\alpha_2 \cdots \alpha_{n+1}$ maps to a valid mCRL2 trace. Hence $b2m(\alpha_1 \cdots \alpha_{n+1}) \in \mathcal{O}_M(Act_{tr}(A))$.

DIRECTION 2 ($\mathcal{O}_M \rightarrow \mathcal{O}'$). Symmetric to Direction 1, replacing $b2m$ with $m2b$ and $\mathcal{O}''$ with $\mathcal{O}_M$. □

4 Verification of Temporal Properties

This section presents the verification framework that bridges Bach's shared-space-oriented temporal logic and mCRL2's μ -calculus. We first introduce the temporal operators supported by Anemone, then introduce the relevant fragment of the μ -calculus used in mCRL2, and finally present the systematic translation of Bach temporal properties into μ -calculus formulas.

4.1 Temporal Logic in Bach and Anemone

The Anemone model checker [12] supports a set of temporal operators oriented towards the shared space contents, making them natural for Bach programs. The basic building block is the predicate $\#x = n$, which checks whether the number of occurrences of si-term x in the shared space equals n . This predicate can be generalized using comparison operators ($=$, $\neq$, $<$, $\leq$, $>$, $\geq$) on si-term occurrences, and further combined using logical connectives (conjunction ($\&$), disjunction ($\mid$), and negation ($!$)) to form composed predicates such as $\#a=1 \ \& \ \#b \geq 2$.

To express richer properties, three core temporal operators are provided:

- **Reach** ϕ : verifies that there exists an execution path reaching a state where ϕ holds in the shared space. For example, $\text{Reach } \#a=1$ checks whether si-term a can eventually be present.
- **Next** ϕ : verifies that ϕ holds in the immediate next state. For example, $\text{Next } \#a=1$ checks whether a appears in the shared space after the very next action.
- ϕ_1 **Until** ϕ_2 : verifies that ϕ_1 holds continuously until ϕ_2 becomes true, and that ϕ_2 eventually holds. For example, $(\#a=0) \text{ Until } (\#b=1)$ asserts that a is absent until b appears.

While this grammar is naturally si-term-oriented, it suffers from significant limitations. The **Reach** operator only proves the existence of a path where a property holds, not its universal establishment across all executions. Furthermore, multiple consecutive **Next** operators and combined **Reach** and **Until** are not permitted. These restrictions make the grammar incapable of verifying complex properties such as liveness or universal invariants, motivating the translation to mCRL2.

4.2 The μ -Calculus Fragment of mCRL2

The mCRL2 toolset uses the modal μ -calculus as its property specification language. Starting from a Linear Process Specification (LPS), the model checker translates a μ -calculus formula into a Parameterized Boolean Equation System (PBES), whose solution determines whether the property holds across all reachable states [11]. Whenever `pbes2bool` returns an answer, it corresponds to a rigorous proof, although termination is not guaranteed for infinite or unbounded state spaces.

The basic modal operators are:

- $[a]\phi$: in every execution of action a , property ϕ must hold (*universal modality*).
- $\langle a \rangle \phi$: there exists at least one execution of action a after which ϕ holds (*existential modality*).
- a^* / a^+ : zero or more / one or more repetitions of a .
- $a_1 . a_2$: sequential composition of action formulae inside a modality, so that $\langle a_1 . a_2 \rangle \phi$ is equivalent to $\langle a_1 \rangle \langle a_2 \rangle \phi$.

These operators combine with logical connectives (conjunction $\&\&$, disjunction $||$, negation $!$) and quantifiers (`forall`, `exists`). The μ -calculus further provides two fixpoint operators, optionally parameterised: $\mu X(p : T = v). \phi$ and $\nu X(p : T = v). \phi$ denote respectively the *least* and *greatest* fixpoint, where the parameter p of type T initialised to v carries state across recursive unfoldings. The least fixpoint expresses liveness (*eventually*), the greatest fixpoint expresses safety (*invariantly*). Boolean expressions are lifted into modal formulae via $\text{val}(\phi)$, which holds iff ϕ evaluates to true.

For instance, $\nu X. ([\text{true}]X \&\& \langle \text{true} \rangle \text{true})$ expresses deadlock freedom — from any reachable state, at least one action is always enabled — while $\mu X. ([\text{true}]X || \langle \text{true}^* . \text{ToId}(t) \rangle \text{true})$ expresses the eventual insertion of a si-term t into the shared space.

4.3 Mapping Temporal Properties

The Bach temporal operators `Reach`, `Next`, and `Until` translate systematically into μ -calculus formulas, as summarized in Figure 5. These translations apply to any formula ϕ , whether it concerns coordination actions or shared space contents.

$$\begin{aligned} \text{Reach } \phi &\longrightarrow \langle \text{true}^* \rangle \phi \\ \text{Next } \phi &\longrightarrow \langle \text{true} \rangle \phi \\ \phi_1 \text{ Until } \phi_2 &\longrightarrow \mu X. (\phi_2 || (\phi_1 \&\& \langle \text{true} \rangle X)) \end{aligned}$$

Figure 5: Translation of Bach temporal operators into μ -calculus.

`Reach  $\phi$` holds if there exists a reachable state satisfying ϕ . `Next  $\phi$` holds if ϕ is satisfied in some immediate successor state. `$\phi_1$  Until  $\phi_2$` holds if either ϕ_2 already holds, or ϕ_1 holds and the property continues in the next state — the least fixpoint ensuring that ϕ_2 is eventually reached.

Content-Aware Verification. A key contribution of this work is the extension of the above translations to properties that directly concern the *contents of the shared space*. This multiset verification dimension appears to be absent from the mCRL2 literature at the time of the research. To support such properties, we

extend the Blackboard process with additional observable actions, triggered whenever specific conditions on the shared space contents are satisfied. For instance, the predicates $\#a = 1$ and $\#b \geq 2$ are encoded as follows:

```
proc Blackboard(myBB: BB) =
  ...
  + sum x: Elem. (count(a, myBB) == 1) -> BB_a_equal_1 . Blackboard(myBB)
  + sum x: Elem. (count(b, myBB) >= 2) -> BB_b_sup_2 . Blackboard(myBB);
```

When ϕ is a predicate over the shared space contents, it is encoded as an observable action prefixed with $BB_$, emitted by the Blackboard whenever the condition holds. The corresponding state formula in the μ -calculus is therefore $\langle BB_ \phi \rangle \text{true}$, which asserts that the action $BB_ \phi$ is available in the current state — and thus, by construction, that ϕ holds. Applying this convention to the general translations of Figure 5 yields:

$$\begin{aligned}
\text{Reach } \#a = 1 & \longrightarrow \langle \text{true}^* \rangle \langle BB_a_equal_1 \rangle \text{true} \\
\text{Next } \#a = 1 & \longrightarrow \langle \text{true} \rangle \langle BB_a_equal_1 \rangle \text{true} \\
(\#a = 1) \text{ Until } (\#b \geq 2) & \longrightarrow \mu X. (\langle BB_b_sup_2 \rangle \text{true} \parallel (\langle BB_a_equal_1 \rangle \text{true} \ \&\& \ \langle \text{true} \rangle X))
\end{aligned}$$

5 Case Study: Synchronous Load Balancer in a Microservice Architecture

5.1 Motivation

Microservice architectures are, by design, distributed systems in which different services may depend on one another. A common coordination challenge in such architectures is the fair distribution of incoming requests across multiple service replicas, in order to avoid overloading a single instance while others remain idle. The Load Balancer pattern addresses this challenge by acting as an intermediary that routes requests to available consumers according to a scheduling strategy.

In this case study, we model a synchronous Load Balancer using a round-robin routing strategy. Each request undergoes a complete processing cycle: the producer initiates the communication, the load balancer forwards the request to a consumer, the consumer responds, the load balancer forwards the response back, and the producer finalizes the cycle. This synchronous request-response pattern is representative of typical production scenarios, for instance in payment service architectures where a response is required to confirm transaction success before initiating subsequent operations such as order confirmation or shipping. We choose the Load Balancer for its conciseness and expressiveness: its round-robin routing and synchronous request-response cycle provide enough behavioral richness to illustrate content-aware verification, while remaining fully presentable within this paper. More complex patterns (Circuit Breaker, Message Broker) are provided as exploratory examples in the associated repository [18].

5.2 Bach Model

The system is initialised as the parallel composition of four agents: `ProducerAgent(pzero, zero, two)`, `LoadBalancer(zero)`, `ConsumerAgent(czero)`, and `ConsumerAgent(cone)`. The producer emits three requests (indexed zero to two) and the load balancer starts at an even round-robin index. The model scales naturally to more producers, consumers, or requests by adjusting the domain definitions. The three agent procedures are given in Figure 6.

```

proc

ProducerAgent(producerId: Service, reqId: MyInteger,
               maxReqId: MyInteger) =
  tell(request(producerId, loadBalancer, reqId));
  sum consumerId in Service:
    get(response(loadBalancer, producerId, consumerId, reqId));
    tell(success(consumerId, producerId, reqId));
    (!(toInt(reqId) = toInt(maxReqId))) ->
      ProducerAgent(producerId, successor(reqId), maxReqId).

LoadBalancer(roundRobinIndex: MyInteger) =
  sum producerId in Service, consumerId in Service,
    reqId in MyInteger:
    (get(request(producerId, loadBalancer, reqId));

    (
      (mod2(roundRobinIndex) = zero) ->
        tell(request(loadBalancer, czero, producerId, reqId));
        LoadBalancer(successor(roundRobinIndex))
    <>
      (mod2(roundRobinIndex) = one) ->
        tell(request(loadBalancer, cone, producerId, reqId));
        LoadBalancer(successor(roundRobinIndex))
    ))
  +
  get(response(consumerId, loadBalancer, producerId, reqId));
  tell(response(loadBalancer, producerId, consumerId, reqId));
  LoadBalancer(roundRobinIndex).

ConsumerAgent(consumerId: Service) =
  sum producerId in Service, reqId in MyInteger :
    get(request(loadBalancer, consumerId, producerId, reqId));
    tell(response(consumerId, loadBalancer, producerId, reqId));
    ConsumerAgent(consumerId).

```

Figure 6: Bach procedures of the synchronous Load Balancer case study. The full specification including data definitions is available in [18].

The case study uses a service domain $\text{Service} = \{\text{pzero}, \text{loadBalancer}, \text{czero}, \text{cone}\}$, a bounded integer type $\text{MyInteger} = \{\text{zero}, \text{one}, \text{two}\}$, and arithmetic helpers: toInt , successor , and mod2 , whose equations are omitted for brevity. To track the routing of requests, messages follow the structure $\text{message}(\text{source}, \text{destination}, \text{context}, \text{id})$, where context preserves the original sender or final recipient throughout forwarding. The load balancer alternates between consumers based on the parity of roundRobinIndex (even $\rightarrow$ czero , odd $\rightarrow$ cone).

Remark 3. successor is defined cyclically with $\text{successor}(\text{two}) = \text{zero}$, ensuring that the load balancer's recursive call $\text{LoadBalancer}(\text{successor}(\text{roundRobinIndex}))$ remains well-defined when roundRobinIndex reaches two. The wraparound has no observable effect: after this final increment,

the producer has already stopped emitting requests (its guard ($\text{reqId} \neq \text{maxReqId}$) halts emission once maxReqId is reached), so the load balancer never processes another request and the wrapped index is never used. Consistency between the bound of `MyInteger` and maxReqId is therefore an important modelling invariant when scaling to larger configurations.

5.3 Generated mCRL2 Model

The Bach specification is automatically translated into mCRL2 by our tool following the framework described in Section 3. The generated code instantiates the Blackboard pattern of Figure 4 with the Load Balancer procedures, wrapped in the standard `allow/comm` synchronization block. The full mCRL2 specification is available in the accompanying repository [18].

5.4 Verification of the Load Balancer Properties

Our automated translation tool fully automates the translation of properties expressible in Anemone — using `Reach`, `Next`, `Until` together with simple counting predicates of the form $\#a = c$ — by generating both the corresponding μ -calculus formula and the Blackboard instrumentation.

The properties of this section, however, exceed Anemone’s expressiveness. Their action-based formulations were written manually in μ -calculus. Their content-aware formulations were either adapted from automatically generated templates and manually edited (No Duplication), or written from scratch while reusing the Blackboard instrumentation patterns produced by the tool for simpler predicates (Absence of Loss) — for example, the single-counter pattern $\#a = c$ provides the structural template from which composed predicates such as $\#a + \#b > 1$ are derived. The tool’s contribution thus extends beyond fully automatic translation: it also provides reusable patterns that reduce the manual effort required for richer properties.

No Duplication of Requests. This property ensures that a given request identifier is never processed twice by the system.

Action-based formulation. The property is expressed by asserting that no execution path contains two consecutive occurrences of the same request identifier:

```
forall reqId: MyInteger.
  !<true* . Told(request(pzero,loadBalancer,reqId)) .
    true* . Told(request(pzero,loadBalancer,reqId))> true
```

Content-aware formulation. To express the property using the Blackboard’s content, a natural first attempt is to extend the Blackboard with an observable action triggered when two occurrences of the same request are simultaneously present:

```
... sum id: MyInteger.
  (count(request(pzero,loadBalancer,id),myBB) > 1)
    -> BB_dup_req . Blackboard(myBB)
```

However, this approach has a fundamental limitation. The Blackboard reflects the *current state* of the shared space at a given moment t , not its full history. A request could be emitted a first time, consumed by the LoadBalancer via `get`, and then emitted again — constituting a genuine duplication — yet the count in the Blackboard would never exceed 1 at any single instant. The property would therefore incorrectly evaluate to `true`. A more robust approach exploits the fact that, in this model, every request eventually leads to a success `si`-term (since no interference is assumed), and crucially, these terms are

never consumed. Duplication can therefore be reliably detected at the level of success si-term, which accumulate in the shared space:

```
... sum id: MyInteger.
    (count(success(czero,pzero,id),myBB) +
     count(success(cone,pzero,id),myBB) > 1)
    -> BB_dup_req . Blackboard(myBB)
```

This yields the following formula, valid under the assumption that each request leads to exactly one success — which holds in this interference-free model but may not generalize to models with interference:

$$!\langle true^* . BB_dup_req \rangle true$$

Cyclic Distribution of Requests. This property ensures that requests are distributed in a strict round-robin fashion between the two consumers. It is naturally expressed using the action-based approach, as it relies on the *order* in which forwarding actions occur rather than on the contents of the shared space at a given moment. The Blackboard emits observable actions whenever a condition holds on its current contents, independently of when the corresponding data was inserted. Verifying alternation therefore requires tracking the *sequence* of observable forwarding actions, which is precisely what the action-based μ -calculus excels at.

```
nu X(last: Service = cone) .(
  [!(exists id: MyInteger.
    Told(request(loadBalancer, czero, id)) ||
    Told(request(loadBalancer, cone, id))
  )] X(last)
  &&
  (forall id: MyInteger.
    [Told(request(loadBalancer, czero, id))]
    (val(last == cone) && X(czero))
  )
  &&
  (forall id: MyInteger.
    [Told(request(loadBalancer, cone, id))]
    (val(last == czero) && X(cone))
  ))
```

Figure 7: μ -calculus formula for the Cyclic Distribution property.

As shown in Figure 7, the formula uses a parameterised fixpoint $nuX(last)$ whose parameter records the consumer targeted by the last forwarding action, with initial value *cone* so that the very first forwarding is constrained to go to *czero*. The formula combines three conjuncts. The first states that any action which is *not* a forwarding to a consumer preserves the invariant without changing *last*. The second states that any forwarding to *czero* requires the previous forwarding to have been towards *cone*, and updates *last* to *czero*. The third is symmetric: any forwarding to *cone* requires the previous forwarding to have been towards *czero*, and updates *last* accordingly. Together, these three clauses enforce strict alternation along every execution.

This example illustrates a general principle: content-aware verification via Blackboard predicates is most effective for *state-based* properties (e.g., presence or count of data), while action-based reasoning is more suited to *ordering* and *sequencing* properties. The two approaches are therefore complementary.

Absence of Loss. This property ensures that every request sent by the producer eventually leads to exactly one successful response, and that no request is silently lost or indefinitely blocked.

Action-based formulation. The action-based formulation is given in Figure 8. The outer νX enforces the property as a global invariant. Upon each request, the inner νY checks that a success is eventually observed, while $\langle \text{true} \rangle \text{true}$ ensures the system never deadlocks before reaching it:

```
forall reqId: MyInteger.
  nu X. (
    [true] X
    &&
    [Told(request(pzero, loadBalancer, reqId))]
      nu Y. (
        [!Told(success(czero, pzero, reqId))
         && !Told(success(cone, pzero, reqId))] Y
        && <true> true
      )
  )
)
```

Figure 8: Action-based formulation of the Absence of Loss property.

Content-aware formulation. We extend the Blackboard with an observable action emitted whenever exactly one success message for a given request is present:

```
... sum id: MyInteger.
  (count(success(czero, pzero, id), myBB) +
   count(success(cone, pzero, id), myBB) == 1)
  -> BB_success(id) . Blackboard(myBB)
```

The condition `== 1` simultaneously encodes two guarantees: the request has been processed (`count  $\geq 1$`) and has not been duplicated (`count  $\leq 1$`). Compared to the action-based formulation, the nested fixpoint νY is replaced by a direct reachability formula $\langle \text{true}^* . \text{BB_success}(\text{reqId}) \rangle \text{true}$, which we argue makes the temporal structure of the property more directly readable — at the cost of relying on the Blackboard instrumentation introduced earlier:

```
forall reqId: MyInteger.
  nu X. (
    [true] X
    &&
    [Told(request(pzero, loadBalancer, reqId))]
      <true* . BB_success(reqId)> true
  )
)
```

5.5 Detecting Violations

Beyond confirming expected properties, the framework also helps a developer pinpoint genuine bugs in the model. To illustrate this, suppose the round-robin logic of the load balancer is accidentally broken — for example, by hard-coding every forwarding to `czero`. Verifying the cyclic distribution property on this faulty variant returns `false`, together with a counter-example trace whose forwarding actions are:

```
... Got(request(pzero, loadBalancer, zero))
Told(request(loadBalancer, czero, pzero, zero))
... Got(request(pzero, loadBalancer, one))
Told(request(loadBalancer, czero, pzero, one))  <- violates alternation
```

The trace is expressed entirely in the Bach vocabulary preserved by our translation (Section 3): the actions `ToId`, `Got`, and the `si`-terms appear exactly as they were written in the original Bach program. Filtering on forwarding actions reveals the violation directly: two consecutive routings to `czero` indicate the broken alternation, without any need to inspect the μ -calculus formula. The cognitive cost of μ -calculus is thus concentrated at specification time, when properties are first written; once the property is in place, diagnosing a violation typically reduces to reading a sequence of familiar Bach actions, which is comparable to inspecting a regular program trace and requires little to no μ -calculus expertise.

6 Conclusion and Discussion

This paper presents a framework for the automated verification of Bach coordination programs using mCRL2. Starting from a formal translation of Bach into mCRL2, we establish a trace correspondence theorem guaranteeing the observable behavior of a Bach agent is faithfully preserved in its mCRL2 encoding. Building on this foundation, we introduce a content-aware verification approach that extends the standard action-based μ -calculus with predicates directly expressed over the shared space contents, by instrumenting the Blackboard process with dedicated observable actions.

Both contributions are supported by an accompanying tool. The translation interface lets the user enter a Bach program in a text area, click *Translate*, and obtain the corresponding mCRL2 specification, with a side panel summarising Bach syntax and coordination primitives for reference. The proof translation interface accepts a Bach temporal property as input and automatically generates the four mCRL2 artifacts required for content-aware verification: the action declaration (`act`), the Blackboard branch (`proc Blackboard`), the allow update, and the μ -calculus formula. Both interfaces are available in the accompanying repository [18].

The Load Balancer case study illustrates the complementary aspects of the two verification styles. Action-based formulations naturally capture *ordering* and *history-based* properties — such as cyclic distribution, where alternation between consumers must be tracked over time. Content-aware formulations excel at *state-based* properties over persistent data — such as absence of loss, where accumulated success messages provide a reliable witness. The no-duplication property further illustrates that naive content-aware encodings may fail when `si`-terms are consumed, but can be recovered by shifting observation to persistent effects. A useful guideline emerges: content-aware verification is most effective for state-based properties over persistent data, while history-based properties are more naturally captured through action-based reasoning.

Beyond verifying expected properties, the framework also supports the practical task of *detecting errors in the model*. When a property fails, the mCRL2 model checker returns a counter-example trace expressed entirely in the Bach vocabulary preserved by our encoding — the original primitives (`ToId`, `Got`, `Asked`, `Nasked`) and the original `si`-terms. As shown on the broken round-robin variant (Section 5.5), a Bach developer can often localise the fault directly from the trace, without inspecting the μ -calculus formula. The cognitive cost of μ -calculus is thus concentrated at *specification time*, when temporal properties are first written, while *diagnosis time* remains close to reading a familiar Bach execution.

However, the content-aware approach has inherent limitations. Instrumenting the Blackboard introduces additional observable actions and increases the state space, which may impact verification performance on larger models. Furthermore, some content-aware encodings rely on model-specific assumptions — such as the absence of interference — that may not hold in more general settings. Such instrumentation should therefore be applied with care.

Future Work. Several directions remain open. On the theoretical side, the trace correspondence established in this paper can be strengthened by investigating whether a weak bisimulation holds between a Bach agent and its mCRL2 encoding, which would provide a finer behavioral characterization of the translation. On the practical side, while this paper introduced a verification approach based on a shared-space model, its empirical exploration remains limited, and a more systematic evaluation of its strengths and limitations on richer coordination patterns remains an open challenge. In particular, applying the framework to more complex patterns such as the Circuit Breaker and the Message Broker would be a natural extension of this work. A third direction concerns the expressiveness of the temporal language itself: extending Anemone’s operators and our automated translation to richer patterns would reduce the manual effort currently required for properties beyond Anemone’s expressiveness, as illustrated by some of the Load Balancer properties of Section 5. Finally, although it addresses a coordination language of a different nature, named Reo, which relies on a control-based approach as opposed to the data-based approach used by Bach, the article [15] proposes other techniques for translating service compositions into mCRL2. A comparison between these techniques and those developed in the present paper would be worthwhile, particularly in light of the relationships already highlighted in the articles [14, 16].

7 Acknowledgment

The authors thank the University of Namur for its support. They also thank the Walloon Region for partial support through the Ariac project (convention 210235) and the CyberExcellence project (convention 2110186).

References

- [1] M. Barkallah & J.-M. Jacquet (2023): *On the Introduction of Guarded Lists in Bach: Expressiveness, Correctness, and Efficiency Issues*. In C. Aubert, C. Di Giusto, S. Fowler & L. Safina, editors: *Proceedings 16th Interaction and Concurrency Experience (ICE) 2023*, EPTCS 383, pp. 55–72, doi:10.4204/EPTCS.383.4.
- [2] Manel Barkallah: *On Reasoning about Socio-Technical Systems: the Multi-Bach Coordination Model and its Workbench Anemone*.
- [3] Jan A. Bergstra & Jan Willem Klop (1985): *Algebra of communicating processes with abstraction*. *Theoretical computer science* 37, pp. 77–121.
- [4] Nicholas Carriero & David Gelernter (1989): *Linda in context*. *Communications of the ACM* 32(4), pp. 444–458.
- [5] D. Darquennes, J.-M. Jacquet & I. Linden (2018): *On Multiplicities in Tuple-Based Coordination Languages: The Bach Family of Languages and Its Expressiveness Study*. In G. Di Marzo Serugendo & M. Loret, editors: *Proceedings of the 20th International Conference on Coordination Models and Languages, Lecture Notes in Computer Science* 10852, Springer, pp. 81–109, doi:10.1007/978-3-319-92408-3_4.
- [6] Rocco De Nicola, Gian-Luigi Ferrari & Rosario Pugliese (2002): *KLAIM: A kernel language for agents interaction and mobility*. *IEEE Transactions on software engineering* 24(5), pp. 315–330.
- [7] Rocco De Nicola, Diego Latella & Mieke Massink (2005): *Formal modeling and quantitative analysis of KLAIM-based mobile systems*. In: *Proceedings of the 2005 ACM symposium on Applied computing*, pp. 428–435.
- [8] David Gelernter & Nicholas Carriero (1992): *Coordination languages and their significance*. *Communications of the ACM* 35(2), p. 96.
- [9] Gelernter, David (1985): *Generative communication in Linda*. *ACM Transactions on Programming Languages and Systems (TOPLAS)* 7(1), pp. 80–112.

- [10] Jan Friso Groote & Mohammad Reza Mousavi (2014): *Modeling and analysis of communicating systems*. MIT press.
- [11] Groote, Jan Friso and Mathijssen, Aad and Reniers, Michel A and Usenko, Yaroslav S and van Weerdenburg, Muck (2009): *Analysis of distributed systems with mCRL2. Process Algebra for Parallel and Distributed Processing*.
- [12] J.-M. Jacquet & M. Barkallah (2021): *Anemone: A workbench for the Multi-Bach coordination language*. *Science of Computer Programming* 202, p. 102579, doi:10.1016/J.SCICO.2020.102579. Available at <https://doi.org/10.1016/j.scico.2020.102579>.
- [13] J.-M. Jacquet & I. Linden (2007): *Coordinating Context-aware Applications in Mobile Ad-hoc Networks*. In T. Braun, D. Konstantas, S. Mascolo & M. Wulff, editors: *Proceedings of the first ERCIM workshop on eMobility*, The University of Bern, pp. 107–118.
- [14] J.-M. Jacquet, I. Linden & D. Darquennes (2018): *On the Relation Between Control-Based and Data-Based Coordination Languages*. In F.S. de Boer, M.M. Bonsangue & J. Rutten, editors: *It's All About Coordination - Essays to Celebrate the Lifelong Scientific Achievements of Farhad Arbab*, Lecture Notes in Computer Science, Springer, pp. 86–106.
- [15] N. Kokash, C. Krause & E.P. de Vink (2010): *Data-aware Design and Verification of Service Compositions with Reo and mCRL2*. In S.Y. Shin, S. Ossowski, M. Schumacher, M.J. Palakal & C.-C. Hung, editors: *Proceedings of the 2010 ACM Symposium on Applied Computing (SAC)*, ACM, pp. 2406–2413.
- [16] C. Krause, Z. Maraïkar, A. Lazovik & F. Arbab (2011): *Modeling Dynamic Reconfigurations in Reo using High-level Replacement Systems*. *Science of Computer Programming* 76(1), pp. 23–36.
- [17] Reuters (2025): *Amazon says AWS cloud service back to normal after outage disrupts businesses worldwide*. Available at <https://www.reuters.com/business/retail-consumer/amazons-cloud-unit-reports-outage-several-websites-down-2025-10-20/>. Accessed: 27 Dec. 2025.
- [18] C. Reuther & J.-M. Jacquet (2026): *Bach-to-mCRL2 Translation Tool and Case Studies*. <https://github.com/UNamurCSFaculty/anemone-bach-to-mcrl2>.